
Geant4のマルチスレッドモード

Geant4 11.3 準拠

Geant4講習会資料-Appendix: HEP/NP/Space 2025



大学共同利用機関法人
高エネルギー加速器研究機構

本資料に関する注意

- 本資料の知的所有権は、高エネルギー加速器研究機構およびGeant4 collaborationが有します
- 以下のすべての条件を満たす場合に限り無料で利用することを許諾します
 - 学校、大学、公的研究機関等における教育および非軍事目的の研究開発のための利用であること
 - ・ Geant4の開発者はいかなる軍事関連目的へのGeant4の利用を拒否します
 - このページを含むすべてのページをオリジナルのまま利用すること
 - ・ 一部を抜き出して配布したり利用してはいけません
 - 誤字や間違いと疑われる点があれば報告する義務を負うこと
- 商業的な目的での利用、出版、電子ファイルの公開は許可なく行えません
- 本資料の最新版は以下からダウンロード可能です
 - <http://geant4.kek.jp/lecture/>
- 本資料に関する問い合わせ先は以下です
 - Email: lecture-feedback@geant4.kek.jp

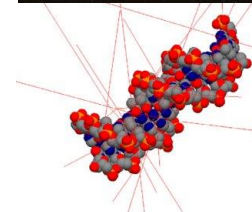
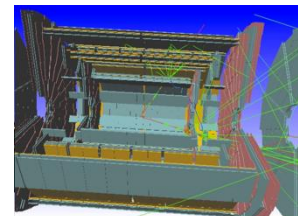
目次

1. はじめに
2. 並列処理の基礎知識
3. Geant4のマルチスレッドモード概観
4. シーケンシャルからマルチスレッドモードへのプログラム移行の注意点
5. Appendix

はじめに

Geant4と並列計算処理の関係

- Geant4を用いた大規模・精密シミュレーションが様々な分野で使用されるとともに、以下の実現が強く求められてきた：
 - シミュレーションの高速化
 - メモリー使用量の削減
- 近年のCPU (Processor)の高速化技術の動向
 - 1990年～2000年間、CPU処理能力はムーア則に従い向上 ← Geant4も'free lunch'を享受
 - 2000年の初めには、CPU処理能力を単独で向上させる技術は、CPUの発熱問題などで限界となる
 - 以後、CPUに複数コア(マルチコアCPU)を搭載して総合的に処理能力を上げるのが主流
 - ← 個人向けPCでも4～16コアは当たりまえ
 - マルチコアCPUを使い処理能力を向上させるには、プログラム作成で並列処理技術が必須となる
- Geant4アプリケーションでも、マルチコアCPUをいかに効率的に使うかが重要
 - Geant4はV.10.0よりマルチスレッド処理機能が使用可能で、マルチコアCPU対応
 - ただし、マルチスレッド処理は、種々ある並列計算処理技法の中の一つ
- この講義ではG4アプリを作成するとき、マルチコアCPUをどのようにすれば効率的に使えるかを述べる
 - ← なお、受講に対して、並列処理、マルチスレッドなどの事前知識は不必要である。



並列計算処理の基礎知識

- マルチコアCPUをGeant4で有効利用するには、並列計算処理の基本的な知識が必要
- 並列計算処理分野の用語使用は混乱があり、それが概念の理解を難しくしている
- 以下では、本講義で使う用語、およびそれらの必要最小限の知識をまとめる

並列計算処理とは

■ 並列計算処理の定義

- 計算処理プログラムを分割し、複数のプロセッサで並列に処理すること

■ 並列計算処理の目的

- 処理の高速化
- 必要メモリの節約（計算資源の節約）

[注] この二つの目的は必ずしも両立するとは限らない

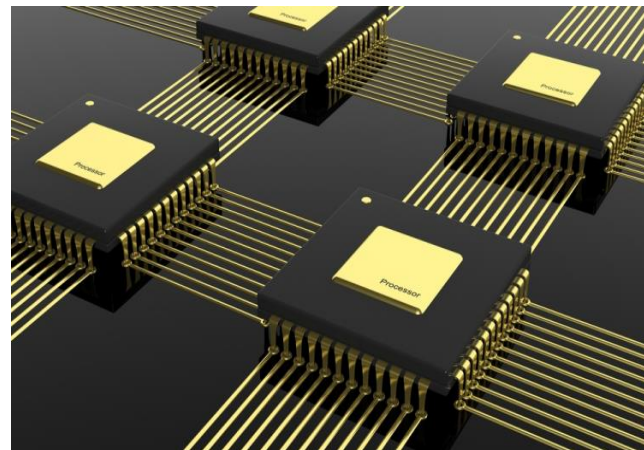
✓ 並列化で実現したい**目的の明確化が重要**

■ 並列計算処理採用の判断基準

- 計算処理で並列処理ができる部分が存在するか？
 - ✓ 存在しなければ並列処理する意味がない
- 存在するならその割合はどの程度か？
 - ✓ 割合が小さければ並列化のメリットは少ない（Appendix: アムダールの法則を参照）
 - ✓ I/O処理の割合が多い場合は並列化のメリットは少ない
- 並列化することで、新たなペナルティが無いか？
 - ✓ 排他処理などの余分な処理が生じ、処理時間が消費されてしまえば意味が無い（後述）

■ 並列処理でメリットを得るのはその実現方法に関して十分な検討が必要

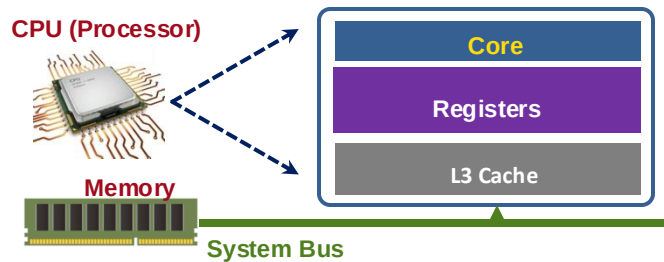
- 漠然と並列処理を使用しても目的を達成できる保証は全くない



<https://www.shutterstock.com>

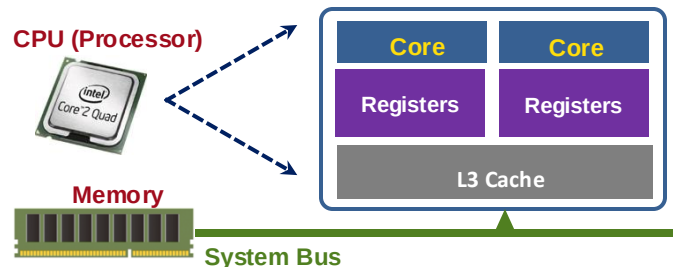
マルチコア CPU (プロセッサ) 概観

■ シングル コア CPU: 一つのコアを1つのCPU(Processor)に集積



- * **Core**: 演算ユニット(ALU)、コントロールユニット など
- * **Registers**: プロセスの状態を保持する制御レジスタ、アドレスレジスタなど
- * **L3 Cache**: Level 3のキャッシュ メモリ (L1, L2キャッシュはCore内にある)

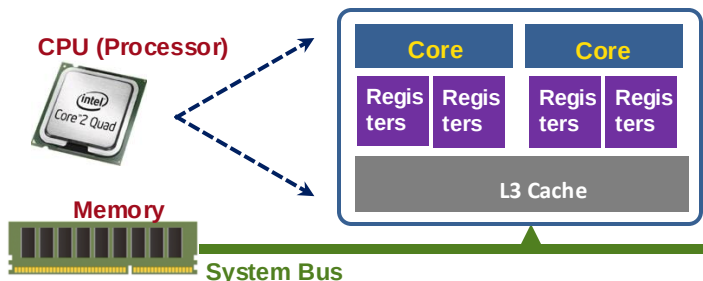
■ マルチ コア CPU: 複数のコアを1つのCPU(Processor)に集積 (下図では、coreは2だが、最近では4~16は当たり前)



- * PCにはマルチコアCPUが通常一つ
- * workstationなどにはマルチコアCPUが複数

[注] ハイパースレッドは、先の講義で述べるプログラミング技術でのスレッドとは全く関係ないものであることに注意

■ ハイパー スレッド: 一つのコアはRegistersを2つ持ち、複数のコアを1つのCPU(Processor)に集積



- * **ハイパースレッド**とは、一つのコアに2つのregistersを持たせて、coreが2倍に増えたように見せる技術
- * Intelチップではハイパースレッド技術とよばれ、AMDチップではSMT(Simultaneous Multi-Threading)とよばれる
- * 左図の場合、2つのコア(物理コア)が、ユーザからは4つのコア(論理コア)に見える: ただし、これによる処理能力の向上は、アプリにもよるが、2倍よりかなり低くなる
- * CPUのコア数は通常、論理コアの数で数える

マルチコアの使用モデル：ParallelismとConcurrency

■ Parallelism とは

- マルチコアCPUのコア数と同じ(or それ以下) 数のプログラムを、個々のコアに独立に割り当て、**物理的に同時に**実行するモデル
- 一つのプログラム実行を、通常、一つの**プロセス**^[注]処理とよぶ
- 一つのプロセスはその処理が完了するまで、コア及びメモリを占有するので**処理速度は速い** — ただし、**メモリ使用率は大きい**

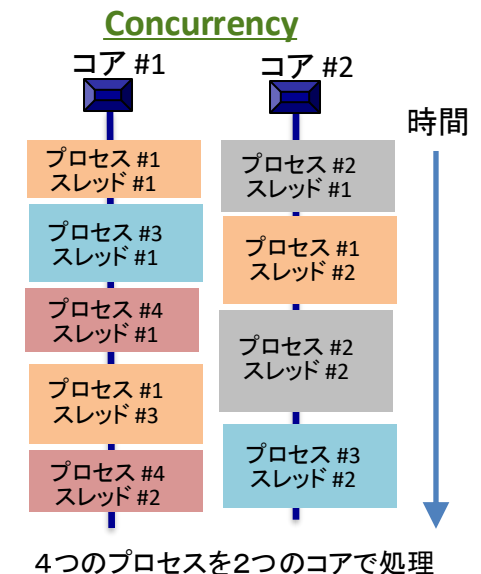
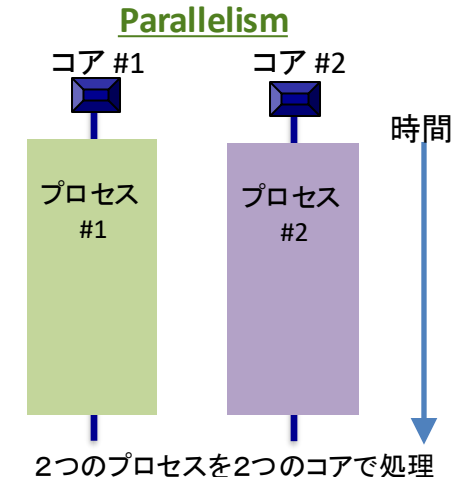
➡ 数値計算、シミュレーションなど**number-crunching**処理に
応用される

[注] プロセスとスレッドの詳細は
次スライドで述べる

■ Concurrency とは

- マルチコアCPUのコア数より多い**プロセス**を、スイッチングで全コアに割り振り、**擬似的に同時に**実行させるモデル
- 実際には、一つの**プロセス**は**スレッド**^[注]とよばれる単位に細分化され、OSが持つスケジューラがコアに**スレッド**を短時間に切り替えロードし、多数の**プロセス**が同時実行されているように見せる
- 複数のスレッドがコア及びメモリを共有するので、スイッチング、排他処理などで**処理速度は遅い** — ただし、**メモリ使用率は小さい**
- ただし、プロセスが処理中に種々の独立機能(例: GUI、ファイルI/O、ネットワーク接続など)を必要とする場合、**このモデル採用は不可欠**

➡ オフィスソフトやweb browserなど、ユーザ応答が必要な日常的
PCアプリケーションに必須な技術



プロセスとスレッド

■ プロセスとは

- プログラム実行開始時にメモリに作られる処理単位
- 必要リソース(スタック、ヒープ、データ等) がメモリに割当られる
- 処理完了まで、一つのコアを占有する
- 新たなプロセス生成をすると、新たにスタック、ヒープ、データ等のリソースが必要となるので、メモリ消費量が多い

[例]

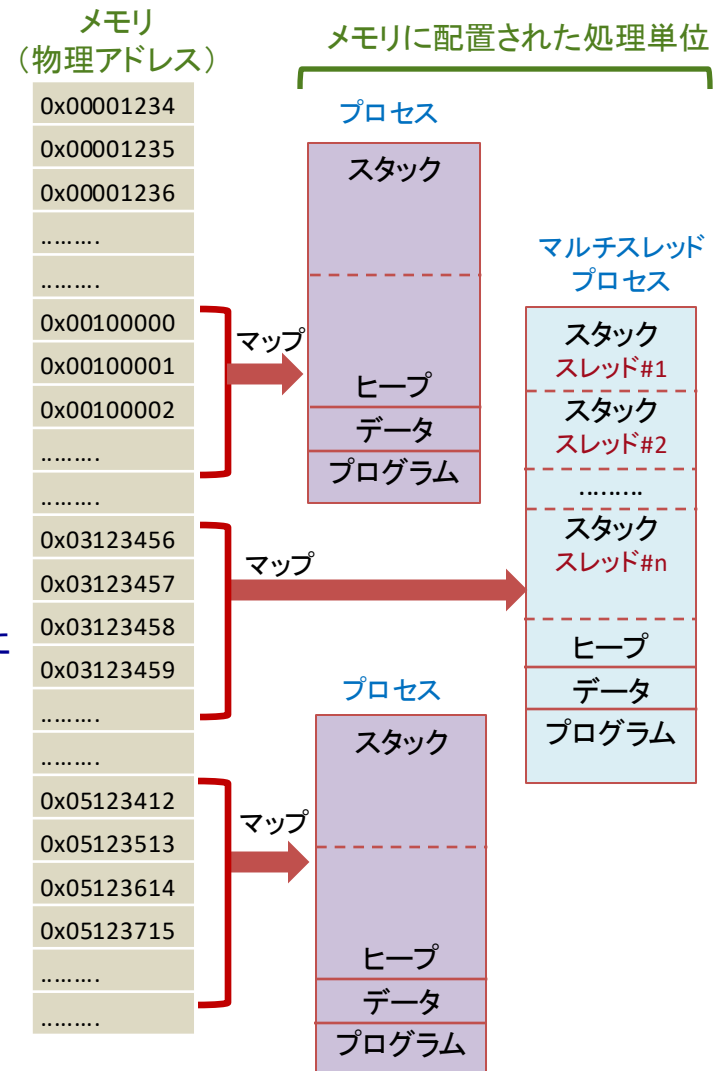
- ユーザが作成した数値処理、シミュレーションなどのプログラム
- ウェブブラウザ、エディタ、表計算などのアプリ

■ スレッドとは

- プロセスに複数の独立処理可能な部分があるとき、それぞれをスレッドとよぶ
- 複数スレッドを同時に処理することをマルチスレッド処理とよぶ
- プロセスは処理開始時に、それぞれのスレッドにスタックを独立に割り当てるが、ヒープ、データは全スレッドで共有するように設定
- 処理開始時に必要となるリソースはスタックのみなので、スレッドの新たな生成は高速で、軽量プロセスともよばれる
- 処理完了まで、一つのコアを占有する
- プロセスは最低1つのスレッド(シングルスレッド)を持つ

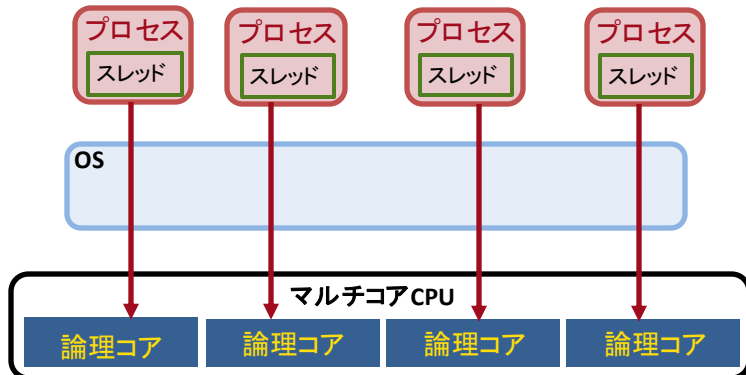
[例]

- ウェブブラウザの内部機能としての'ユーザI/O処理'、'ネットワーク・アクセス'などはそれぞれスレッドで実装される



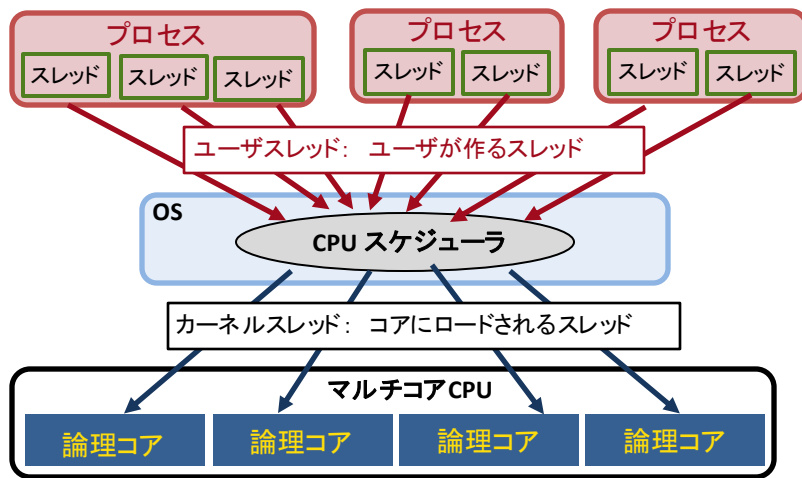
Parallelism、Concurrencyモデルの実装と特徴

■ Parallelismモデルの実装 — マルチプロセス処理



- 個々のコアに単独のプロセスを割り当て、同時に処理する
- 個々のプロセスは互いに独立なリソースを持って進行するので、一つのプロセスが他のプロセスを干渉することはない
- プロセス間で情報交換が必要な場合、例えば MPI (Message Passing Interface) を使用して、通信処理が可能であるが、**排他処理**が必要となり実装は複雑となる
- プロセス間で情報交換が不要の場合、単独のプロセス処理と同じになるので、実装は単純となる
 - ✓ この実装は 'Embarrassingly Parallel' とよばれる
 - ✓ 排他処理のない、安全な並列処理が実装可能
 - ✓ 数値処理、シミュレーションなどを高速処理するのに最適

■ Concurrencyモデルの実装 — マルチスレッド処理



- コア数より多いスレッドを、スイッチングで全コアに割り振り、スレッドの親プロセスを**擬似的に複数同時に処理**させる
- 各スレッドは共有メモリ領域にアクセスできるので、**排他制御が必須**
 - ✓ **排他制御**は**スレッドセーフ**ともよばれ、複数のスレッドが共有領域(ヒープ等)アクセスで競合(レーシング)し、データ不整合が生じること防ぐプログラム技術
 - ✓ **排他制御**の安易な使用は処理時間を著しく低下させる (詳細はAppendixの「マルチスレッドでの排他制御」を参照)
- スレッド実装は左図の様に**カーネルス/ユーザスレッド**に分かれる
- 二つのスレッド間のマッピング及びスイッチングの処理法はOSに依存
- この処理の実装及びデバッグには技術経験が必須

Geant4アプリをマルチコアCPU処理する時に考慮すべきこと

■ 採用できる処理手法は2つある：

1. Parallelism — マルチプロセス処理
2. Concurrent — マルチスレッド処理

どちらを採用するかは、処理したいプログラムの特徴で決める

■ プログラムが必要とするメモリサイズが小さい(≦ 1GB)場合：

- Parallelism(マルチプロセス)処理の採用を考える
➡ 'Embarrassingly Parallel' 処理が最適
- Geant4の旧来の手法で処理プログラムを作成することが可能
- Geant4アプリケーションの多くは、この処理で並列化が可能
- メモリ不足の問題が出たら、その時点でConcurrent処理への移行検討を行う

■ プログラムはメモリサイズが大きい(≧ 10GB)場合：

- Concurrent処理(マルチスレッド)処理の採用を考える
- この場合、排他処理なしでプログラムを作成できるかどうか、技術的な困難性を決める鍵となる
- 排他処理の利用が不可避の場合、マルチスレッド処理に十分な理解なしに、プログラム作成は困難

Geant4のマルチスレッドモード概観

以下の講義は、Geant4でマルチコアCPU処理にマルチスレッドモード(concurrency)を使いたいユーザのために用意されている

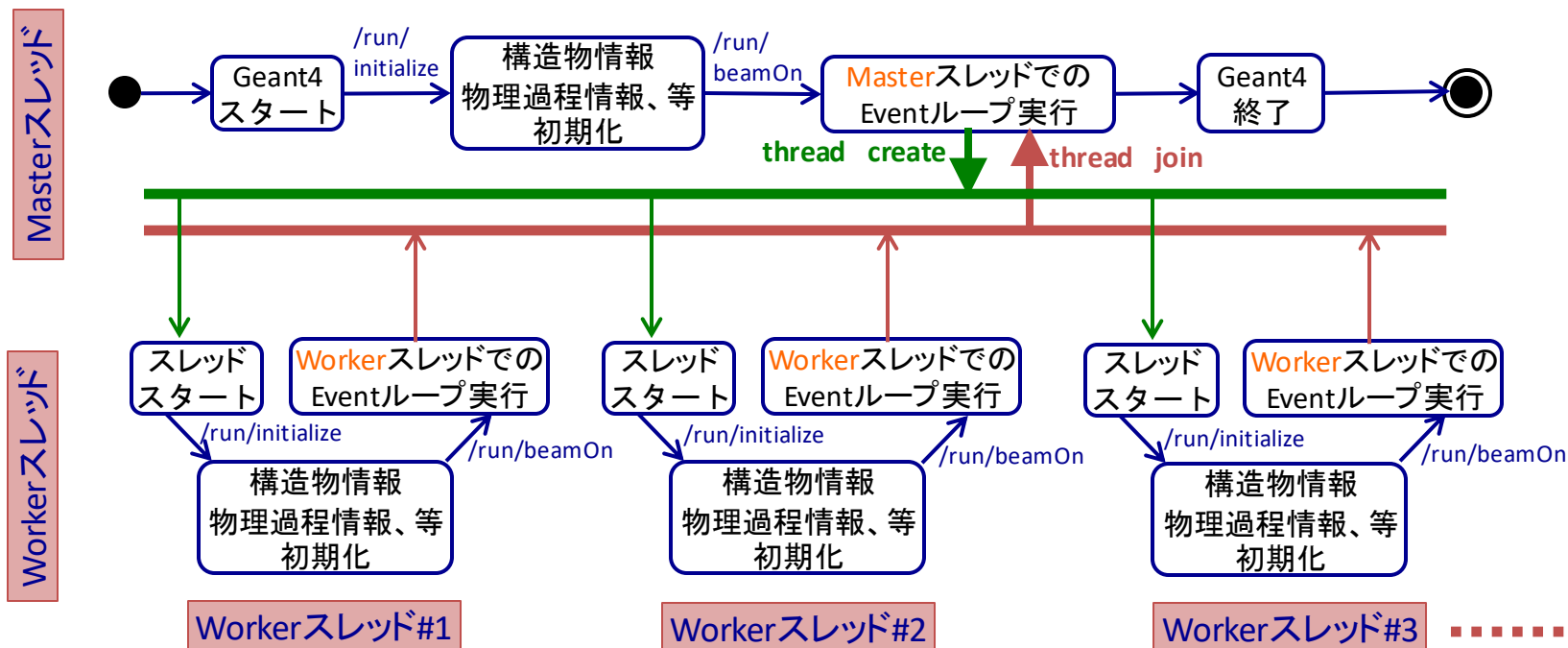
Geant4におけるマルチスレッド処理の基本デザイン

■ Event独立性を用いた並列処理

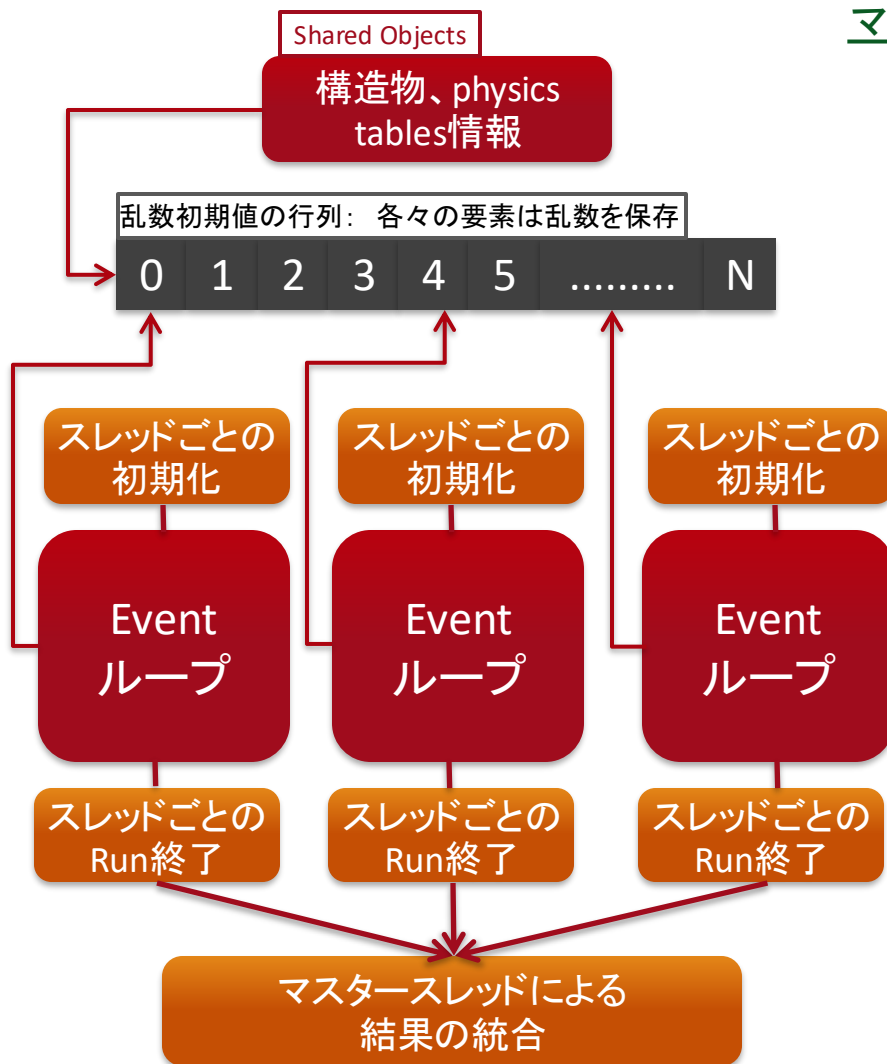
- Geant4は与えられたEventを独立にシミュレートするので、それぞれのEventは並列に処理できる

■ Master-Workerモデルに基づくデザイン

- シミュレーション全体を管理する部分(Masterスレッド)と個々のeventを処理する部分(Workerスレッド)とに分離
- Masterスレッドは複数のWorkerスレッドを生成し、event処理の実行を任せる



Geant4マルチスレッド処理の基本デザイン - つづき



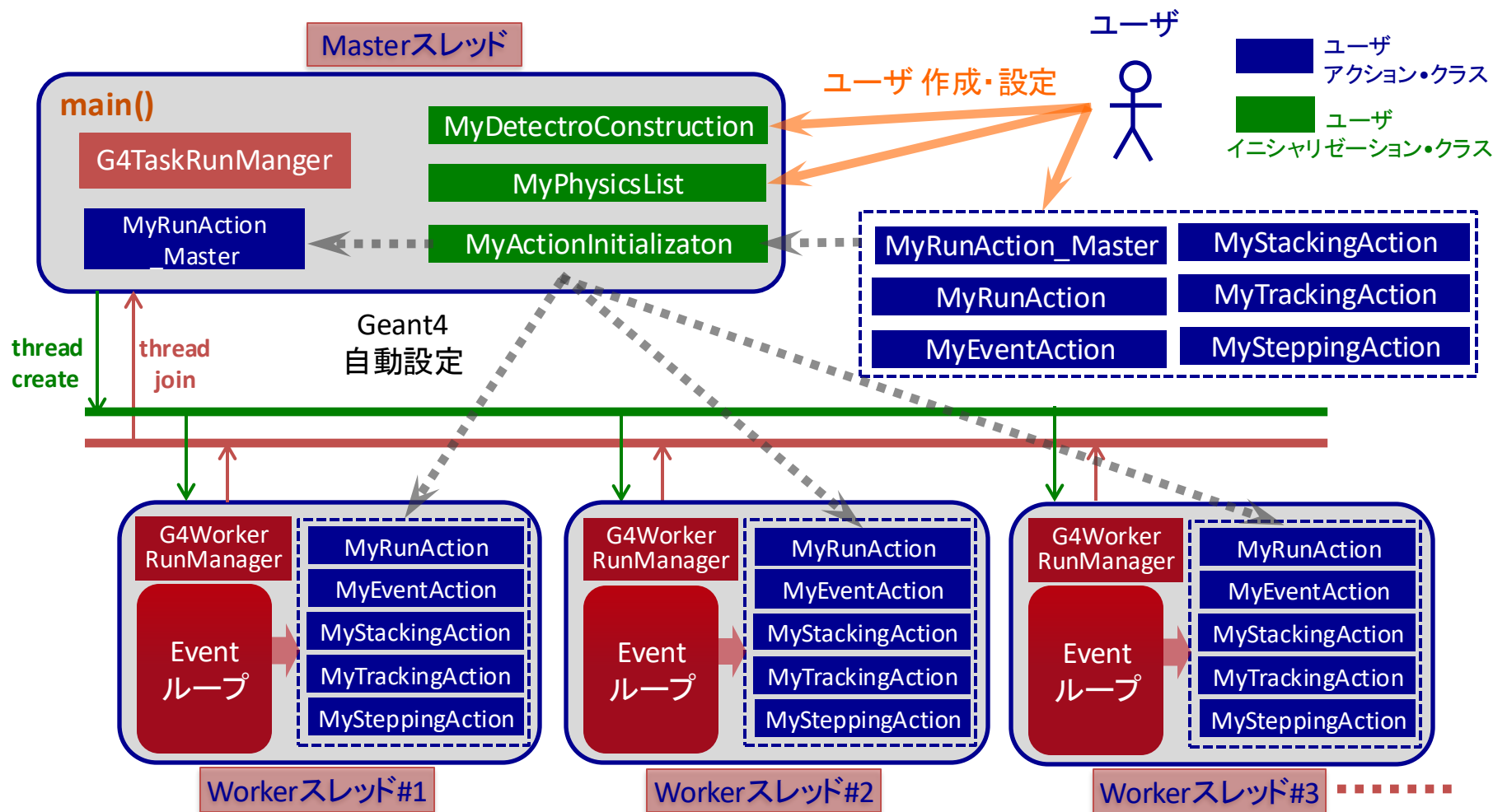
マルチスレッド処理の基本的な流れ

- シミュレーション開始前にMasterスレッドが、各Workスレッドが使う乱数初期値を作る
- また、メモリを大量に消費する情報はMasterスレッドがShared Objectsとして作成し、Workerスレッドはこれを共有する：
 - 構造物情報、physics tablesなど
 - アプリケーションのサイズを小さくできる
- Workerスレッドが作られると、そこで新しいG4Runが作られ、event処理が開始される
- Workerスレッドで得られた処理結果はそれぞれのスレッドが保存：
 - Sensitive Detector, hitsなど
- それぞれのスレッドで保存された情報は、スレッドの終了時にMasterスレッドにまとめる
 - コマンドラインのスコアリング、あるいはevent処理結果をG4toolsで扱っている場合、自動的に情報はまとめられる
 - それ以外の場合、ユーザがMasterスレッドへの情報伝達の手法をプログラムする

[注] スレッドが処理途中にMasterスレッドに情報を伝達可能だが、処理速度を大きく低下させる可能性がある

ユーザ・アクションのマルチスレッドにおける扱い

- Geant4でユーザが作成するクラス／オブジェクトがマルチスレッド環境でどのように扱われているかを理解しておくことが、実際のマルチスレッド・アプリケーション構築に不可欠



シーケンシャルからマルチスレッドモードへの プログラム移行の注意点

以下の講義は、すでにシーケンシャルモードで稼働しているプログラムをマルチスレッドモード(concurrency)に移行する手順を述べる

移行手順の概要

- Geant4アプリのシーケンシャル・モードとマルチスレッド・モード
 - シーケンシャル・モード: マルチスレッド ライブラリを使用せずに作成されたアプリ (伝統的なモード)
 - マルチスレッド・モード: Geant4が提供するマルチスレッド・ライブラリを使用して作成されたアプリ
- Version 11.0以降は上記二つのモードを同時にサポートする標準Geant4ライブラリを提供
 - ユーザはどのモードを使うかでライブラリを切り分ける必要はない
- 今までのシーケンシャル・モードのアプリをマルチスレッドに移行するには、以下の手順をとる
 1. アプリのmain()で、マルチスレッドモードのrun managerを選択する
 2. User Action Initializationで必須オブジェクトを設定(geometry, physicslistなど)する
 2. アプリが 'sensitive detector' あるいは磁場を使う場合、'ConstructSDandField()' に設定する
 3. Workerスレッドで得られた個別シミュレーション結果をまとめる処理プログラムを書く
 - この処理はスコアリングともよばれる
 - この部分のプログラム作成はマルチスレッドへの移行で最も難しいところ

[注] 以下の講義では、上記手順の具体的な説明を行う。
また、Hands-onでプログラムを作成して実行してみる

main()プログラムの移行

■ main()のマルチスレッド・モードとシーケンシャル・モードでの違いは以下の点のみ

- シーケンシャル : *G4RunManager*を使う
- マルチスレッド : *G4RunManagerFactory*を使う
 - デフォルトでマルチスレッドモードになる
(run managerのタイプは*G4TaskRunManager*が選択される)
 - スレッドの数は*GSetNumberOfThreads()*で設定する

シーケンシャル モード

```
//+++++
// Geant4 Application: Tutorial ...
//+++++

#include "Geometry.hh"
#include "UserActionInitialization.hh"

#include "G4RunManager.hh"
#include "G4UImanager.hh"
#include "G4VisExecutive.hh"
#include "G4UIExecutive.hh"
#include "FTFP_BERT.hh"

//-----
int main( int argc, char** argv )
//-----
{
// Construct a run manager
auto runManager = new G4RunManager;

//.....
```

変更

変更

マルチスレッド モード

```
//+++++
// Geant4 Application: Tutorial ...
//+++++

#include "Geometry.hh"
#include "UserActionInitialization.hh"

#include "G4RunManagerFactory.hh"
#include "G4UImanager.hh"
#include "G4VisExecutive.hh"
#include "G4UIExecutive.hh"
#include "FTFP_BERT.hh"

//-----
int main( int argc, char** argv )
//-----
{
// Construct a run manager
auto runManager =
    G4RunManagerFactory::CreateRunManager();
    runManager->SetNumberOfThreads(4);
```

G4RunManagerFactoryを使ってrunManagerを作る:

- マルチスレッド モード環境になる
- run managerタイプは*G4TaskRunManager*になる

G4VUserActionInitializationの移行

- マルチスレッドモードではユーザアクション登録の際に、以下の様に2種類の‘build’を使い分ける必要がある
 - **Build()**: Workerスレッド用
 - ✓ シーケンシャルモードで使っていたのと同じ関数
 - ✓ PrimaryGeneratorAction, RunAction, EventActionなどを登録する
 - **BuildForMaster()**: Masterスレッド用
 - ✓ マルチスレッドモードでMasterスレッドにRunActionを登録したい時に用いる
 - ✓ 登録が不必要ならこの関数は使う必要がない
 - 下のコードではMasterスレッドとWorkerスレッドに異なったのRunActionを登録している

UserActionInitialization.hh

```
//-----  
class UserActionInitialization : public  
    G4VUserActionInitialization  
//-----  
{  
public:  
    UserActionInitialization();  
    virtual ~UserActionInitialization();  
  
    virtual void Build() const override;  
    virtual void BuildForMaster() const  
        override;  
};
```

UserActionInitialization.cc

```
void UserActionInitialization::Build() const  
{  
    SetUserAction(new PrimaryGeneratorAction);  
    SetUserAction(new RunAction);  
    SetUserAction(new EventAction);  
    SetUserAction(new SteppingAction);  
}  
  
void UserActionInitialization::BuildForMaster() const  
{  
    SetUserAction(new RunActionForMaster);  
}
```

シーケンシャルモード
でのUserActionと同じ

MasterスレッドにRunActionを
登録したいときは、ここに書く

構造物定義 (Detector Construction)の移行

- マルチスレッドでGeant4が扱うオブジェクトは、以下の 2 種類に分類できる
 - 共有オブジェクト : MasterとWorkerスレッドで共有される
 - スレッドローカルオブジェクト : それぞれのWorkerスレッドで独立に存在し、相互干渉はない
- [注] 情報が共有オブジェクトにあるかスレッドローカルにあるかを知ることは重要
- 構造物の情報はこの 2 種類のオブジェクトの両方に存在し、定義方法が異なる
 - 共有オブジェクト : physical volume, logical volume, material情報、その他
 - 定義で使用する関数 : G4VUserDetectorConstruction::Construct()
 - スレッドローカル : sensitive detector, electro-magnetic field、その他
 - 定義で使用する関数 : G4VUserDetectorConstruction::ConstructSDandField()

DetectorConstruction.hh

```
//-----  
class DetectorConstruction : public G4VUserDetectorConstruction  
//-----  
{  
public:  
    DetectorConstruction();  
    ~DetectorConstruction() override;  
  
    G4VPhysicalVolume* Construct() override;  
    void ConstructSDandField() override;  
  
};
```

[注] ConstructSDandField()を使い、Sensitive DetectorとMagnetic Fieldを定義する方法はHands-Onで学ぶ

Workerスレッドで集積された結果の集約方法

■ 結果集約について

- 結果集約の処理はスコアリングともよぶ
- 結果をどう集約するかはシミュレーションの目的により様々で、統一した手法はない
- ユーザが独自にプログラムするのが基本だが、それにはmulti-threadの基本知識が要求される
- Geant4は簡単な結果集約ツールを用意 — ツールが目的に合致するかを先づ調べることを勧める

■ Geant4が用意する結果集約ツール

- Scorer — 医療応用などで、energy deposit等を自動的に積算する
- G4Accumulable — HEPのRootツールのHistogram/Ntuple技法に基づく結果集約

■ 結果集約(スコアリング)をユーザがプログラムするとき考慮すべきこと

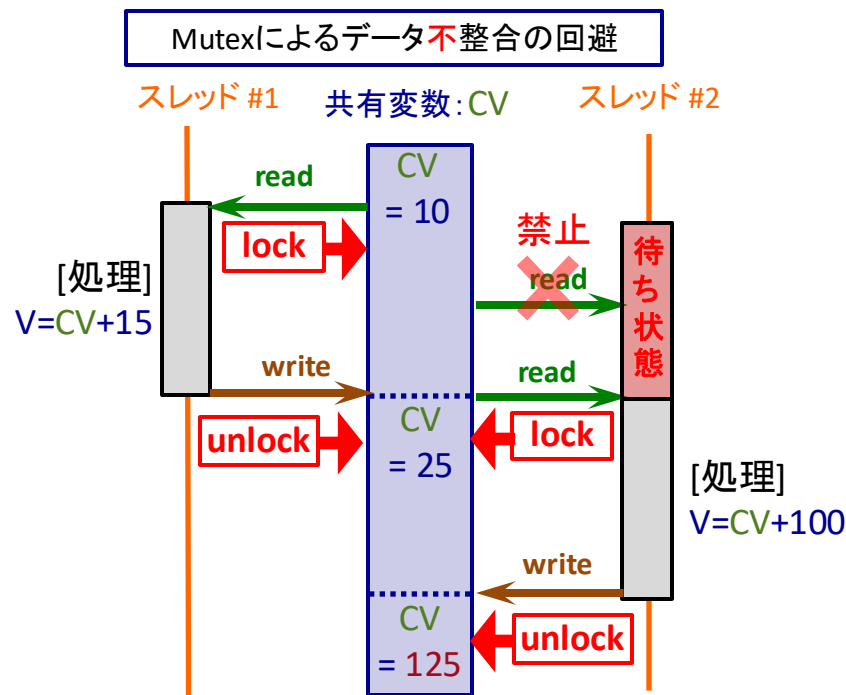
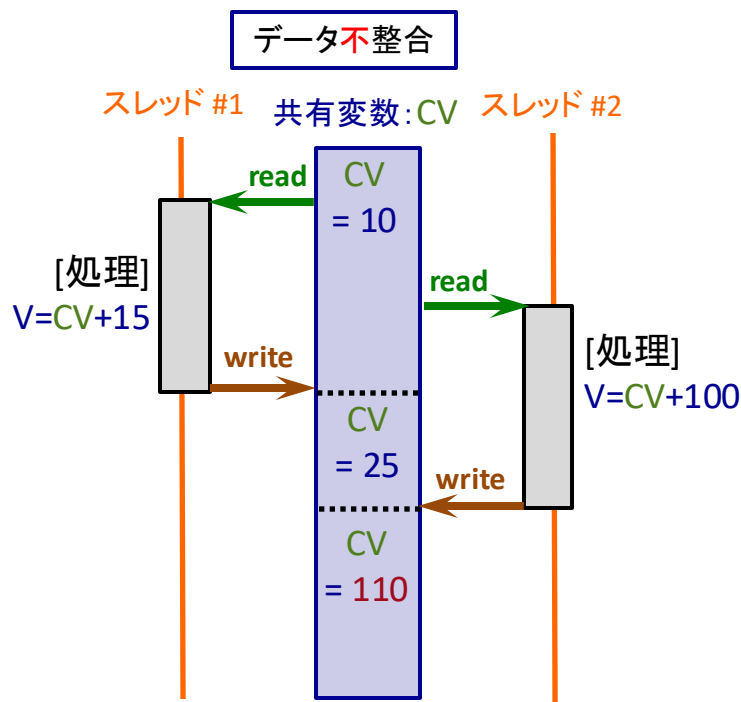
- 排他処理は処理速度の大きなペナルティとなるので、その使用が不可避を見極めるのが鍵となる
- まずは排他処理なしで(ペナルティを最小限に抑えることを含めて)、スコアリングの設計・作成を行う
 - 次のHands-onセッションではこの手法を学ぶ
- 排他処理が不可避の場合は、相談できるマルチスレッド経験者とともに設計・作成を行う

講義 終了

Appendix

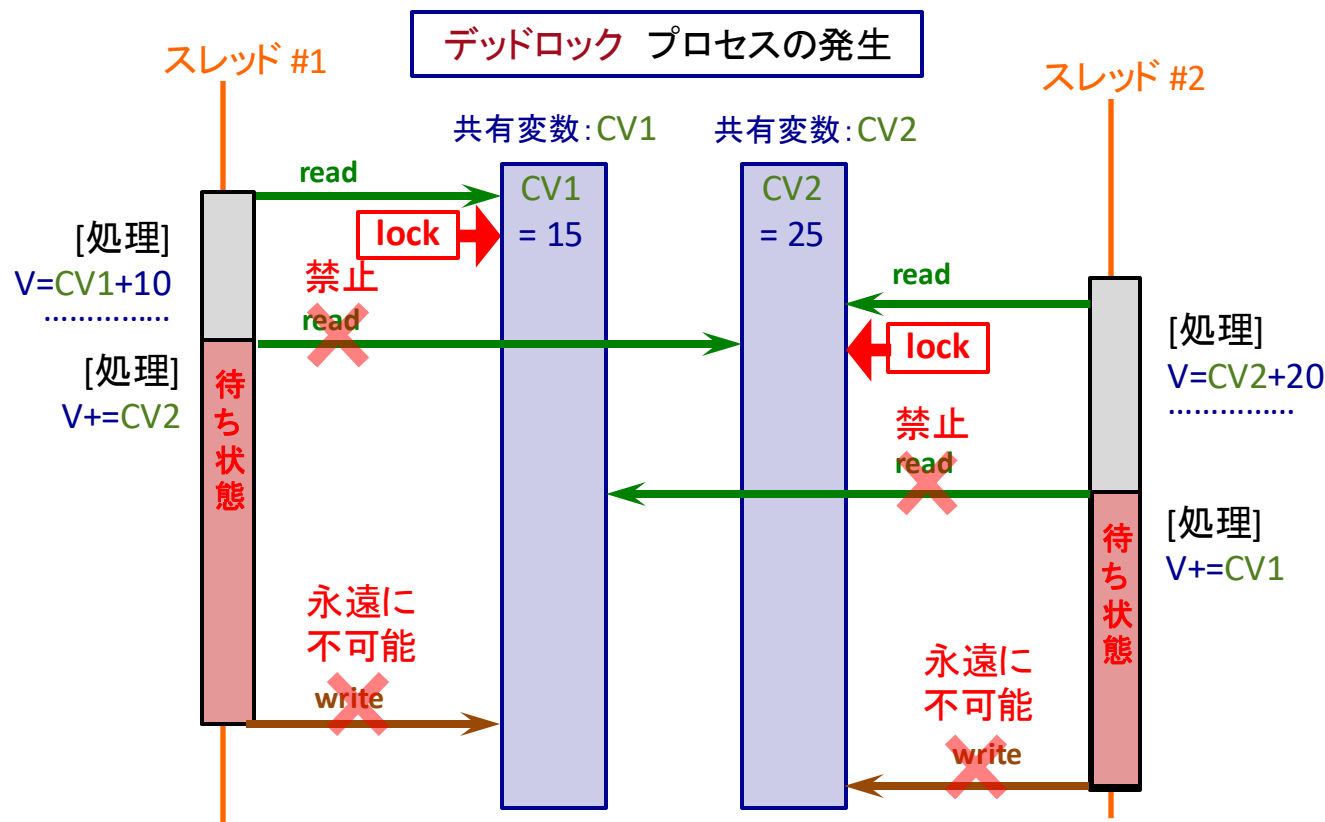
マルチスレッド処理での排他制御 – 1: mutex/atomic

- 排他制御をどう実現するかは、マルチスレッド処理だけでなく、並列計算分野での重要かつ困難な課題である
- 例えば、左下図の様に2つのスレッドが、共有領域を同時変更するとデータ**不整合**が生じる
 - この様なデータ不整合はデバッグは難しい
- 右下図は**Mutex**(**MUT**ual **EX**clusion)とよばれる手法で**不整合**が生じることを回避している
 - C++では**mutex**以外に**atomic**とよばれる回避手法もある
 - コード作成時に不整合が生じる可能性のある部分を見つけ、回避手段を取る必要がある



マルチスレッド処理での排他制御 – 2: デッドロック

- 前のスライドでは、**mutex**を使ったデータの不整合回避を示したが、この回避処理をしても、下図の様にスレッドが複数領域にアクセスする場合、処理全体が**デッドロック**(完全停止)になる可能性がある
- このデッドロックの回避処理手法はなく、プログラム作成時に注意深く、このようなコードを書かない様にするしかない

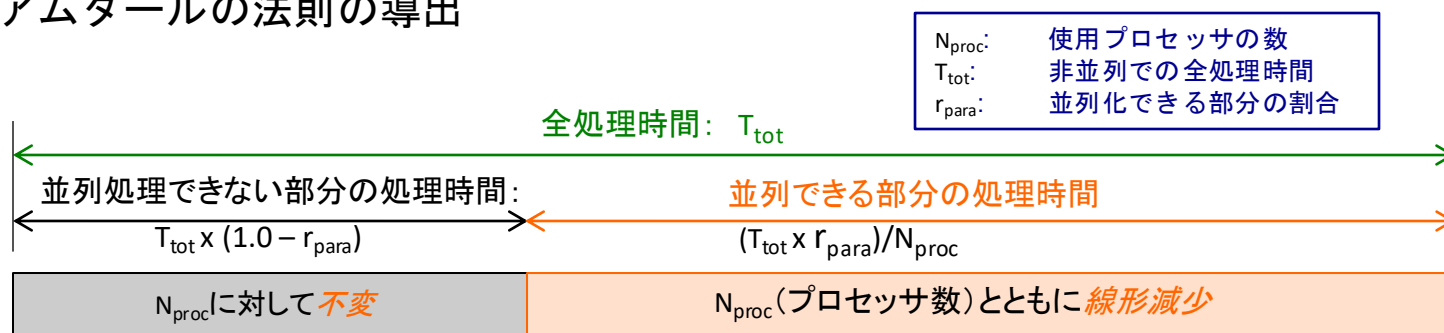


アムダールの法則

■ アムダールの法則での仮定条件

- 並列化の対象となるプログラムは並列化できる部分と並列化できない部分に分かれる
- 並列化できない部分の処理時間はプロセッサ数を増やしても変化しない
- 並列化できる部分の処理時間はプロセッサ数の増加とともに線形減少

■ アムダールの法則の導出

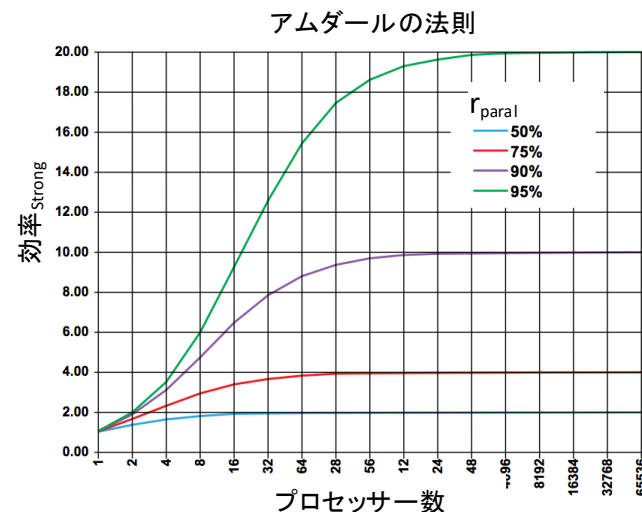


$$\text{効率} \equiv \frac{T_{\text{tot}}}{T_{\text{tot}} \times (1.0 - r_{\text{para}}) + T_{\text{tot}} \times r_{\text{para}} / N_{\text{proc}}}$$

$$= \frac{N_{\text{proc}}}{(1.0 - r_{\text{para}}) \times N_{\text{proc}} + r_{\text{para}}}$$

■ アムダールの法則の考察

- 並列処理不可部分が大きくなると並列処理効率は急激に悪化する



References

■ Geant4 Multi-thread

- “Multi-threading in Geant4: a general introduction”, *Andrea DOTTI*
18th Geant4 Collaboration Meeting, 23-27 September 2013,
Plenary session 3 - Geant4 version 10 (part 1)
<https://indico.cern.ch/getFile.py/access?contribId=3&sessionId=7&resId=0&materialId=slides&confId=250021>
- “Migration of examples and user code to MT mode”, *Ivana HRIVNACOVA, et al.*
18th Geant4 Collaboration Meeting, 23-27 September 2013,
Plenary session 3 - Geant4 version 10 (part 1)
<https://indico.cern.ch/getFile.py/access?contribId=1&sessionId=7&resId=1&materialId=slides&confId=250021>
- “Quick migration guide for Geant4 version 10.0”, *Andrea DOTTI*
<https://twiki.cern.ch/twiki/bin/view/Geant4/QuickMigrationGuideForGeant4V10>